TUTEDUDE

Bash Scripting

Practical Report

Practical	Reading the contents of a protected file using a Bash script
Platform	TryHackMe – TuteDude Cybersecurity Room
Student	Vivek Satpute
Date	5 November 2026

This report records the steps followed, the Bash script used, the terminal output, and the confirmed TryHackMe answer.

1. Objective

To create and execute a Bash script that reads the contents of the protected file **secret.txt** and displays the result in the terminal.

2. Tools and Environment

• TryHackMe TuteDude Cybersecurity room
• Linux terminal accessed through the provided lab environment
• GNU nano text editor
• Bash shell

Room: https://tryhackme.com/jr/tutedude-cybersec

3. Procedure

Step 1 – Check the working directory. The `ls` command was used to view the available files. The directory contained `read_secret.sh`, `secret.txt`, and the supporting lab files.

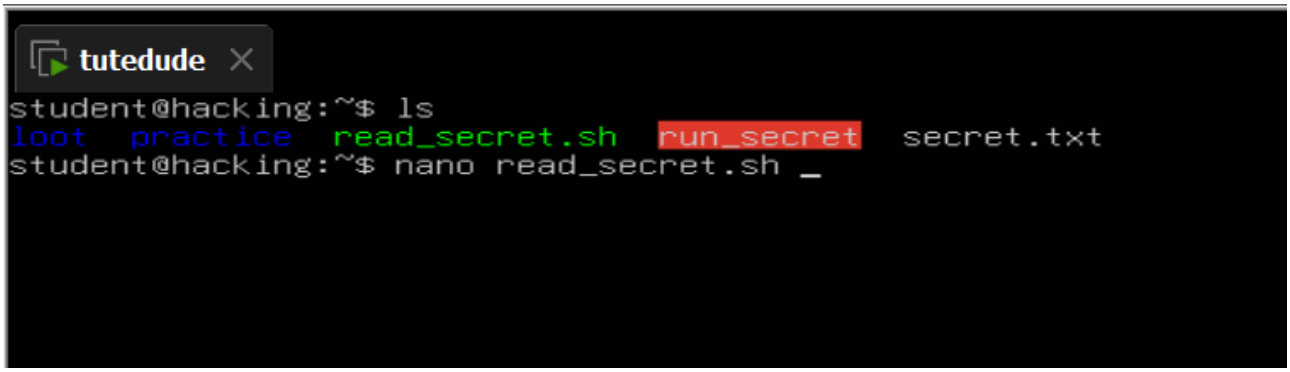


Figure 1. Listing the lab files and opening read_secret.sh in nano.

Step 2 – Edit the script. The file `read_secret.sh` was opened in GNU nano. The script was edited to read the contents of the target file.

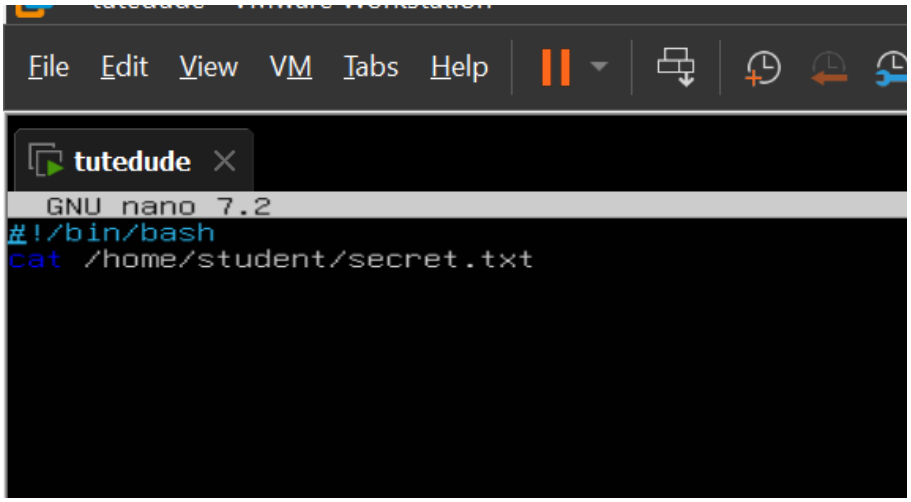


Figure 2. Script content shown in the nano editor.

4. Script and Execution

The terminal evidence shows the following script content:

```
#!/bin/bash
cat /home/student/secret.txt
```

The script was then executed from the terminal. The output displayed the contents of secret.txt.

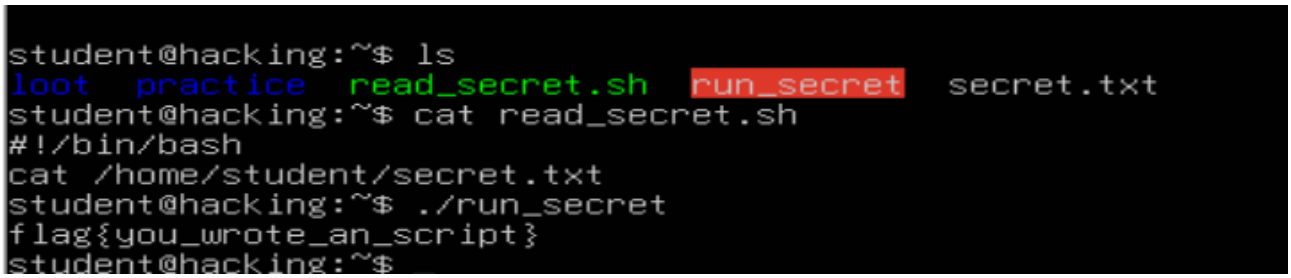


Figure 3. Terminal execution and the returned flag.

5. TryHackMe Result

The answer submitted in the TryHackMe room was accepted and displayed as **Correct Answer**.

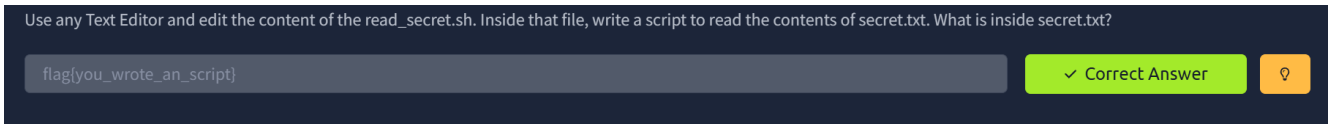


Figure 4. TryHackMe confirms that the submitted answer is correct.

6. Result

The contents of `secret.txt` were successfully read and displayed. The value shown in the terminal and accepted by TryHackMe was:

```
flag{you_wrote_an_script}
```

7. Conclusion

This practical demonstrated how to edit and run a Bash script in a Linux environment, read a file from a specified path, and verify the output against a TryHackMe task. The TryHackMe screenshot provides confirmation that the submitted answer was accepted.

Technical note: The captured script uses `cat` directly. The task handout also asks for the file content to be stored in a variable before printing it. To meet that additional requirement exactly, the script can be written as shown below:

```
#!/bin/bash
secret=$(cat /home/student/secret.txt)
echo "$secret"
```

The variable-based version above is a compliant alternative; it is shown separately and is not represented as the exact code visible in the submitted terminal screenshot.